



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	MS.Swetha Nagasri
Student Name	I.Harshika
Roll Number	2520030607
Branch	CSE
Course Outcome	CO-2

Case Study Topic: Flipkart Fulfillment Center Historical Inventory Tracking

Work Done Summary:

Flipkart's fulfillment center manages inventory for 8 product categories, where stock levels frequently change due to incoming shipments, customer orders, and stock transfers. The analytics team often needs to access historical inventory data, such as checking the total stock available for specific product categories at earlier points in time. Storing a complete inventory snapshot after every update would consume excessive memory and become inefficient for large-scale warehouse operations. To solve this problem, Flipkart uses a Persistent Segment Tree, which creates a new version of the tree after each update while sharing unchanged nodes with previous versions, thereby optimizing memory usage. Initially, the inventory values are [12, 7, 25, 18, 9, 14, 6, 30], forming the first version (V0) of the segment tree. The inventory is then updated sequentially: stock[3] += 50 to create V1, stock[6] -= 4 to create V2, and stock[3] -= 12 to create V3. During each update, only the modified root-to-leaf path creates new nodes, while the remaining nodes are shared with earlier versions. Finally, the system calculates the total inventory for the range stock[3...6] across versions V0, V1, V2, and V3 to analyze historical stock changes efficiently. This case study demonstrates how Persistent Segment Trees provide efficient memory management, faster updates, and quick historical query processing in warehouse inventory systems.

Implementation Details:

1. Created the initial inventory array [12, 7, 25, 18, 9, 14, 6, 30] with indexing starting from 0, and constructed the initial Persistent Segment Tree (V0) for range-sum queries.
2. Applied the first update $\text{stock}[3] += 50$ to create V1, changing the value from 18 to 68. Only the nodes along the modified root-to-leaf path were recreated, while the remaining nodes were shared with V0.
3. Applied the second update $\text{stock}[6] -= 4$ to create V2, updating the value from 6 to 2. Again, only the affected path was recreated, while unchanged nodes were reused.
4. Applied the third update $\text{stock}[3] -= 12$ to create V3, changing the value from 68 to 56, with only the modified path updated and other nodes shared.
5. Performed the range sum query for $\text{stock}[3...6]$ across all versions:
 $V0 = 47$
 $V1 = 97$
 $V2 = 93$
 $V3 = 81$
6. Observed that the Persistent Segment Tree reduced memory usage and enabled efficient historical inventory queries compared to storing full inventory snapshots.

Result/Output Screenshots:

The screenshot shows an IDE with the following components:

- EXPLORER:** A project named "DSA 2 REPORT" containing three files: "HyderabadMetroMST.java", "MediFlowAVL.java", and "PersistentSegmentTree.java".
- Editor:** The "PersistentSegmentTree.java" file is open, showing the following code:

```
1 class PersistentSegmentTree {
2
3     // Node Structure
4     static class Node {
5         int sum;
6         Node left, right;
7
8         Node(int sum) {
9             this.sum = sum;
10        }
11    }
12
13    // Initial Inventory Array
14    static int arr[] = {12, 7, 25, 18, 9, 14, 6, 30};
```
- TERMINAL:** The terminal shows the command to run the program and its output:

```
PS C:\Users\iliha_eyuesh1\OneDrive\Desktop\First Year\Term 3\DSA 2\DSA 2 Report> & 'C:\Program File
\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D
\Users\iliha_eyuesh1\AppData\Roaming\Code\User\workspaceStorage\e40e96f20e973a5233789afcb9e52afb\rec
at.java\jdt_ws\DSA 2 Report_9d771891\bin' 'PersistentSegmentTree'

RANGE SUM QUERY [3...6]

V0 = 47
V1 = 97
V2 = 93
V3 = 81

Persistent Segment Tree created successfully.
PS C:\Users\iliha_eyuesh1\OneDrive\Desktop\First Year\Term 3\DSA 2\DSA 2 Report> 
```

The Persistent Segment Tree efficiently tracked inventory changes without storing complete copies of the inventory after every update. Each update created a new version while reusing unchanged nodes, which saved memory. Historical range sum queries for stock[3...6] were successfully computed across versions V0, V1, V2, and V3, allowing easy comparison of past inventory states.

Time Complexity:

- Update Operation: $O(\log n)$
- Range Sum Query: $O(\log n)$
- Memory per Update: $O(\log n)$

Compared to storing full inventory snapshots, the Persistent Segment Tree provided a more efficient and scalable solution for historical inventory tracking.

GitHub Link : <https://github.com/iliharshika/DSA-2-Project>

Student Signature	Faculty Signature